

Explainable AI: Concepts, Techniques, and Implementations

From Taxonomy to Practical XAI Methods (LIME, SHAP, Grad-CAM, and Beyond)

Week 5 Contents

- Introduction
 - Need for XAI
 - What is an Explanation in AI? ✓
- Techniques of Explainable AI and XAI Methods
 - PCI, Occlusion, SHAP, LIME, Grad-CAM, Guided Backpropagation, CAM. ✓
- Hands-on Implementation of XAI methods

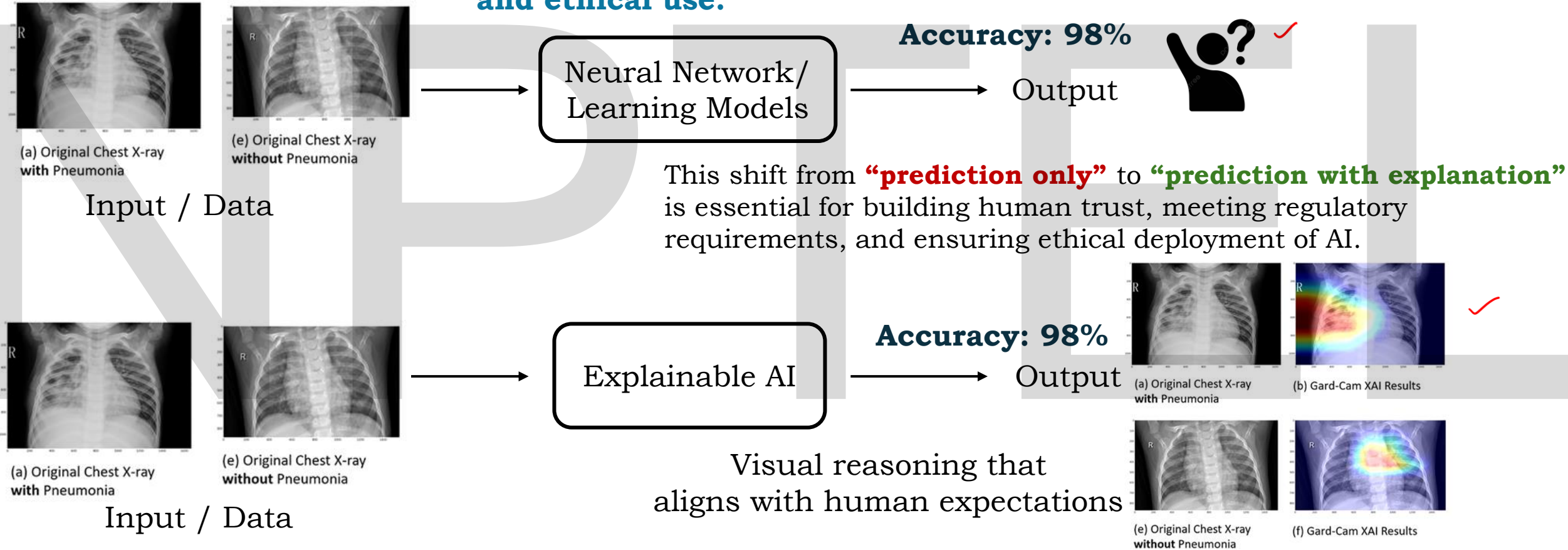
Today's Session Overview

- Introduction to XAI
- XAI methods: PCI, Occlusion in detail

Need for XAI

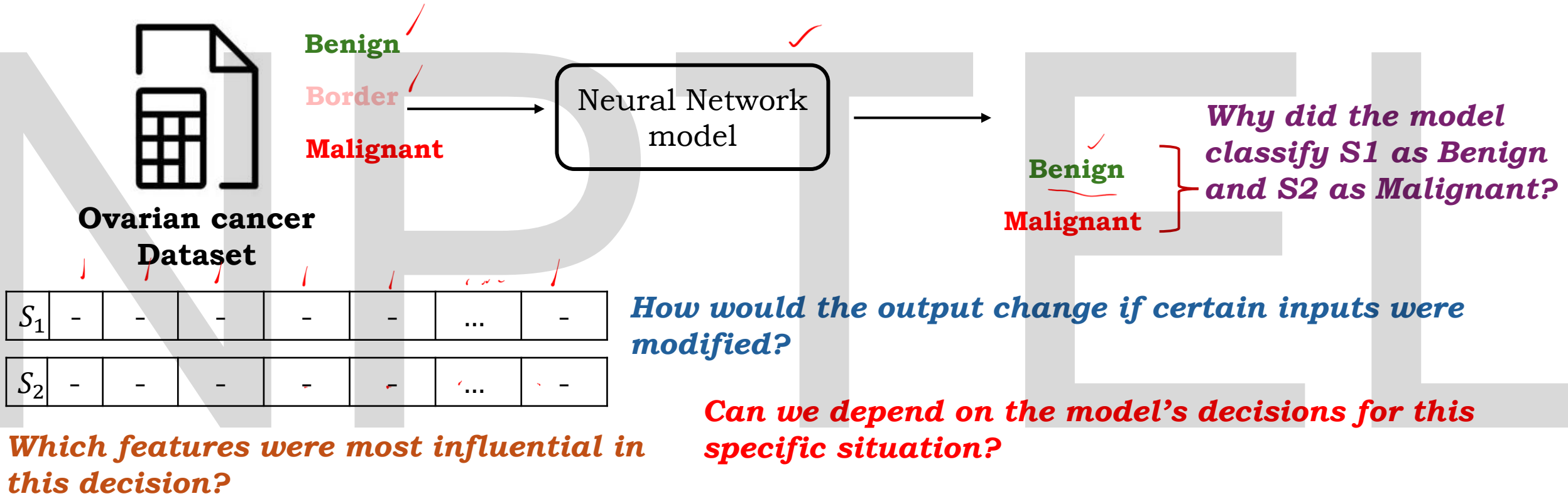
Despite their effectiveness in terms of performance, many of these NN models—often referred to as **“black box” systems**—operate in ways that are not easily understandable by humans.

While these models can achieve high predictive accuracy, their **lack of transparency** raises concerns about **trust, accountability, fairness, and ethical use**.



The need for explainability becomes especially critical in **high-stakes applications**.
Medical diagnosis, autonomous driving, fraud detection, and judicial decision-making, where incorrect or biased predictions can have serious consequences.

What is an Explanation in AI?



An **explanation in AI** generally answers these questions. And answering these questions **is the role of XAI**. Thus makes the decision-making process of AI systems **interpretable, transparent, and trustworthy** for human users.

Techniques of XAI

Permutation-based methods	Post-hoc, Model Agnostic	Gradient-based methods	Interpretability by Design
✓ Permutation Channel Importance ✓ Occlusion ✓ SHAP	✓ LIME	✓ Gradient-weighted Class Activation Mapping (Grad-CAM) ✓ Guided Backpropagation ✓ Guided Grad-CAM ✓ DeepLift ✓ Integrated Gradients	✓ Class Activation Maps (CAMs)

Randomly shuffling input and observing how the prediction changes.

*Methods that generate explanations after training.
Can be applied to any model.*

The NN model computes the gradients during backpropagation. Gradient-based XAI methods use these gradients to highlight the important regions of the input.

There are some models that are inherently transparent when they are built. For such models, we use interpretability-by-design techniques.

Permutation-based methods





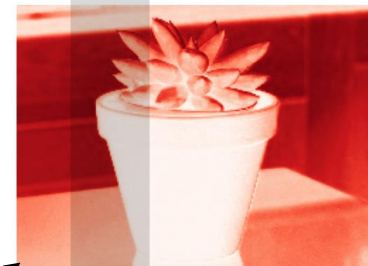
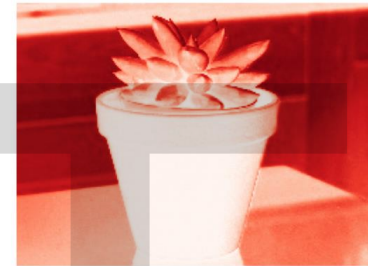
```
model = Sequential([
    Conv2D(32, (3, 3), activation='relu',
    input_shape=(32, 32, 3)),
    MaxPooling2D((2, 2)),
    Flatten(),
    Dense(64, activation='relu'),
    Dense(4, activation='softmax')
])
```

Compute baseline performance

- Evaluate the model on a test dataset.
- Record the baseline accuracy.
Example: Accuracy = 90%.

The model relies heavily on **Red**, than on **Green**, and barely on **Blue**.

$$I(R) = 90 - 70 = 20$$



$$I(G) = 90 - 85 = 5$$



$$I(B) = 90 - 89 = 1$$

- If accuracy drops a lot → Red channel is important.
- If accuracy hardly changes → Red channel wasn't critical.

$$I(ch) = \text{Baseline Performance} - \text{Performance after permutation of channel } ch$$

$$I(R) = \underline{90} - 70 = 20$$

Permutation Channel Importance (PCI):

- PCI tells us which colour channel (Red, Green, or Blue) the CNN is using the most to make its prediction.

How does it work?

Train the CNN model

- Train your image-classification model as usual.

Compute the baseline performance

- Test the model on the original test set.
- Record the accuracy.
- Example: **Baseline accuracy = 90%**.

Pick one colour channel & Permute (shuffle) that channel

- Make a copy of the test images.
- Randomly shuffle only the pixel values of the chosen channel across the images (Red values get mixed up, but Green and Blue stay the same).
- This breaks the useful pattern in that channel.

Re-evaluate the model

- Run the model on these “shuffled-channel” images.
- Compute the new accuracy.
- Example for Red: accuracy after permutation = 70%.

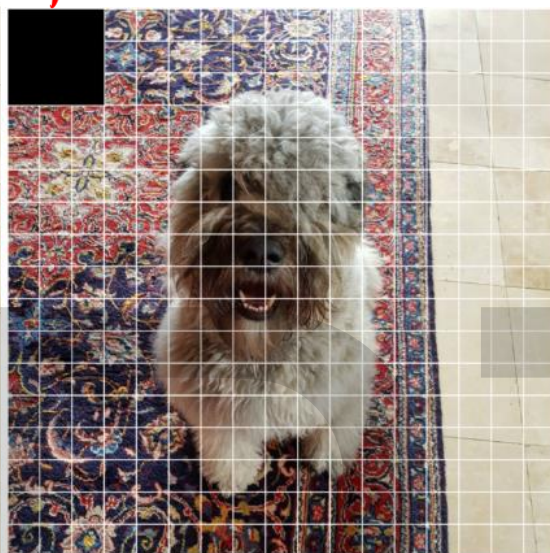
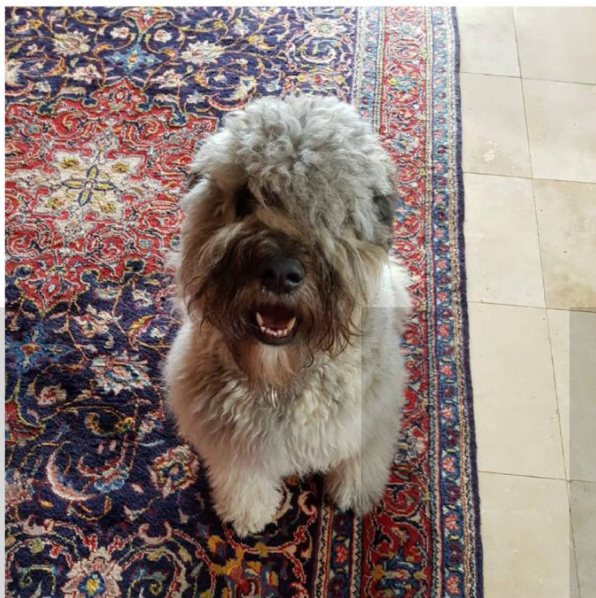
Compute the importance score for that channel

- $I(ch) = \text{Baseline Accuracy} - \text{Accuracy after permutation of channel } ch$

Repeat for every channel (R, G, B) and get $I(R)$, $I(G)$, $I(B)$

Interpretation

- **Big drop in accuracy (large I)** → that channel is important for the model.
- **Small or no drop (small I)** → that channel is less important.



Occlusion

$$3 \times 3$$

0.01	0.01	0.01						
0.01	0.01	0.01						
0.01	0.01	0.01						

3	0	1	2	7	4	2	3	4
1	5	8	9	3	1	7	8	9
2	7	2	5	1	3	6	5	4
0	1	3	1	7	8	2	3	4
4	2	1	6	2	8	6	5	4
2	4	5	2	3	9	8	7	6
5	6	7	8	9	2	3	4	5
2	3	4	5	7	6	5	4	1
9	0	8	7	5	4	3	2	6

0	0	0	2	7	4	2	3	4
0	0	0	9	3	1	7	8	9
0	0	0	5	1	3	6	5	4
0	1	3	1	7	8	2	3	4
4	2	1	6	2	8	6	5	4
2	4	5	2	3	9	8	7	6
5	6	7	8	9	2	3	4	5
2	3	4	5	7	6	5	4	1
9	0	8	7	5	4	3	2	6

*

1	0	-1
1	0	-1
1	0	-1

=

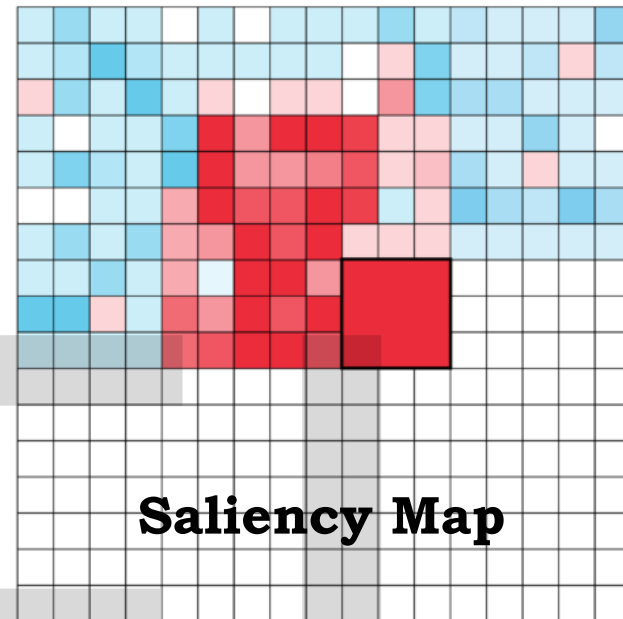
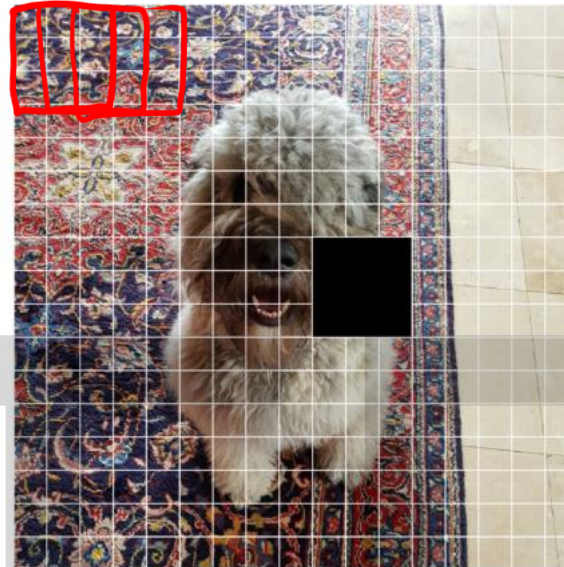
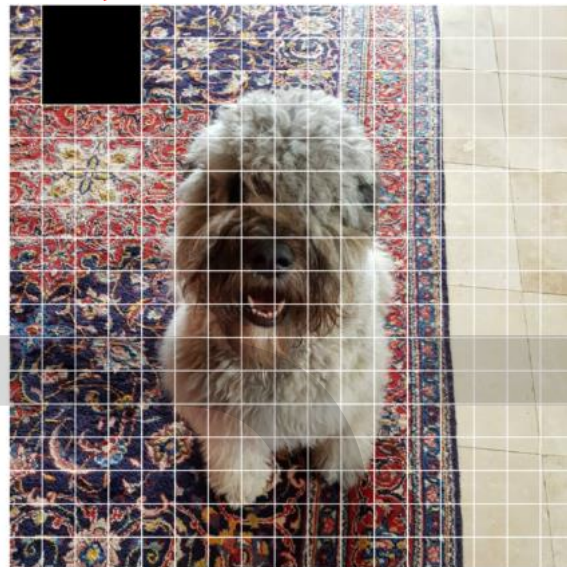
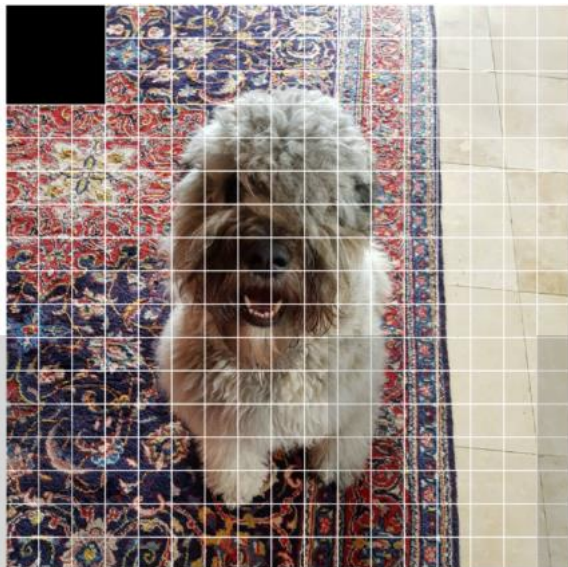
0			

Baseline prediction: model's output probability on the original (non-occluded) image.

Example: $P_{\text{dog_original}} = 0.98$

Occluded prediction: model's output probability on the occluded image.

Example: $P_{\text{dog_occluded}} = 0.97$ $\Delta p = 0.98 - 0.97 = 0.01$



0	0	0	2	7	4	2	3	4
0	0	0	9	3	1	7	8	9
0	0	0	5	1	3	6	5	4
0	1	3	1	7	8	2	3	4
4	2	1	6	2	8	6	5	4
2	4	5	2	3	9	8	7	6
5	6	7	8	9	2	3	4	5
2	3	4	5	7	6	5	4	1
9	0	8	7	5	4	3	2	6

$$\Delta p = 0.98 - 0.97 = 0.01$$

3	0	0	0	7	4	2	3	4
1	0	0	0	3	1	7	8	9
2	0	0	0	1	3	6	5	4
0	1	3	1	7	8	2	3	4
4	2	1	6	2	8	6	5	4
2	4	5	2	3	9	8	7	6
5	6	7	8	9	2	3	4	5
2	3	4	5	7	6	5	4	1
9	0	8	7	5	4	3	2	6

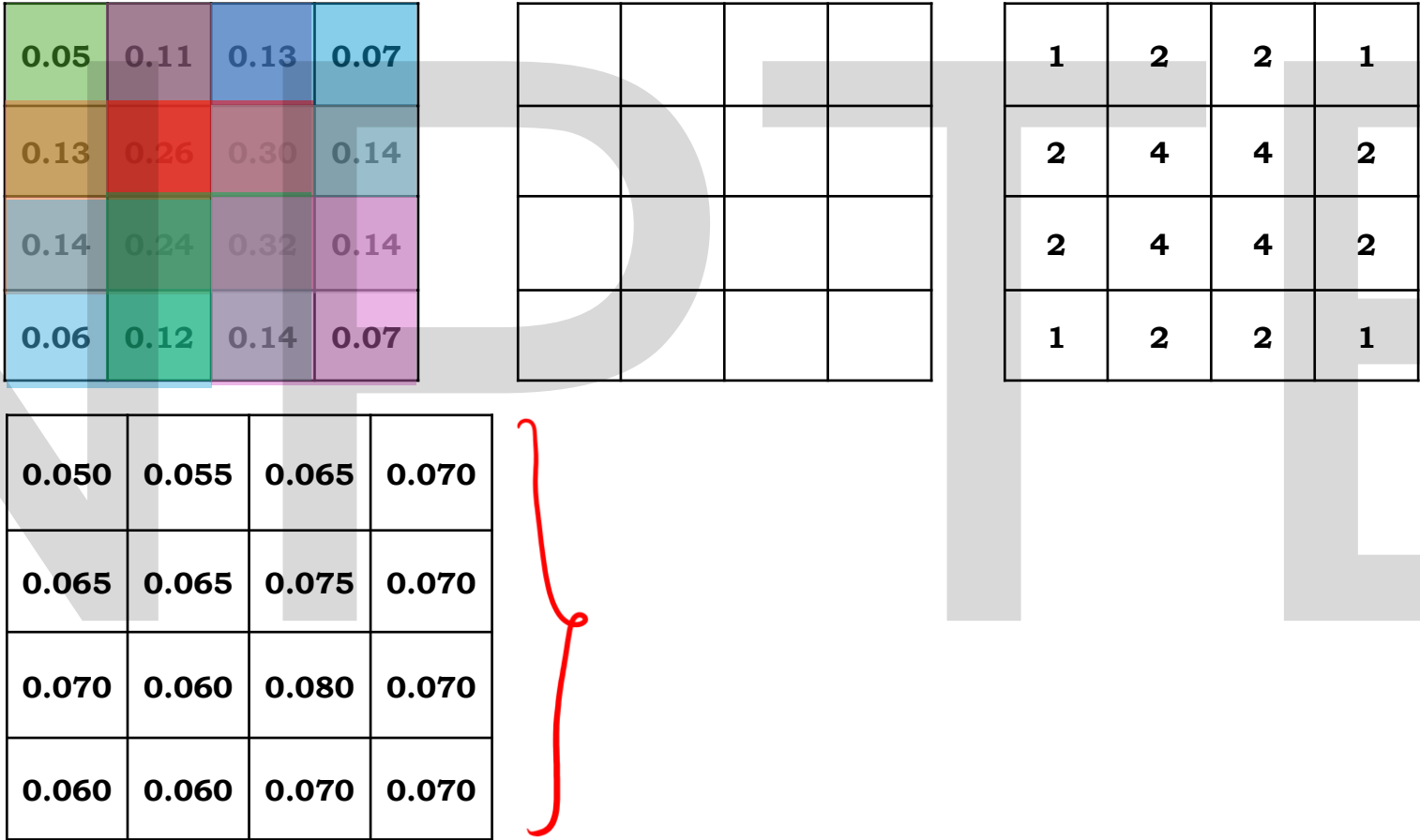
$$\Delta p = 0.98 - 0.97 = 0.01$$

3	0	1	2	7	4	2	3	4
1	5	8	9	3	1	7	8	9
2	7	2	5	1	3	6	5	4
0	1	3	1	0	0	0	3	4
4	2	1	6	0	0	0	5	4
2	4	5	2	0	0	0	7	6
5	6	7	8	9	2	3	4	5
2	3	4	5	7	6	5	4	1
9	0	8	7	5	4	3	2	6

$$\Delta p = 0.98 - 0.42 = 0.56$$

0.01	0.02	0.02	0.01					
0.01	0.02	0.02	0.01					
0.01	0.02	0.02	0.01					
				0.56	0.56	0.56		
				0.56	0.56	0.56		
				0.56	0.56	0.56		

How to normalize the saliency maps



Occlusion

Step 1: Compute the Baseline Prediction

- Feed the **original image** to the model.
- Record the predicted probability for the target class.
(This is the baseline score.)

Step 2: Place an Occlusion Patch on the Image

- Mask a small region (e.g., a **3×3** or **10×10** patch) by replacing it with **0 / black / mean pixel value**.
- Only that region is hidden; the rest of the image stays the same.

Step 3: Predict Again With the Occluded Image

- Pass the occluded image through the model.
- Record how much the probability changes.

Step 4: Compute Importance for That Patch

$$\Delta_i = f(x) - f(x_{[i]})$$

- If **Δ is large**, the masked region is **important** ✓
for prediction.
- If **Δ is small**, the region is **not important**. ✓

Step 5: Slide the Patch Across the Entire Image

- Move the occlusion patch across the image **in a sliding-window manner** (stride = 1 or higher).
- For each position, repeat Steps 2–4.
- Add the Δ value to all pixels covered by the patch.

Step 6: Build the Saliency Map

- Each pixel accumulates importance based on how many times the patch covered it.
- Normalise the map (0–1) because different pixels are occluded different numbers of times.

Step 7: Visualise the Final Heatmap ✓

- Heatmap shows **which parts of the image the model relied on**.

Summary

✓ Why XAI?

- Black-box models lack transparency.
- Important for trust, ethics, fairness, and decision accountability.
- Critical in high-risk domains (healthcare, finance, autonomous systems).

✓ What is an Explanation?

- Answers “*Why did the model predict this?*”
- Identifies important features and evaluates model reasoning.

✓ XAI Techniques Covered Today

- **Permutation Channel Importance (PCI):**
Measures channel-level importance (R/G/B) by shuffling channels.
- **Occlusion Sensitivity:**
A permutation-based pixel-level method that hides small patches and tracks change in prediction.

✓ Key Takeaways

- Both PCI and Occlusion are model-agnostic and simple to implement.
- Large drop in prediction → region/channel is **important**. ✓
- Occlusion produces a saliency heatmap that visually highlights influential regions.

Next Session

SHAP – SHapley Additive exPlanations

LIME – Local Interpretable Model-Agnostic Explanations

How they generate feature importance for individual samples.

NIPTELL

Thank you